



School of Mathematics & Computer Science

Comparison of Numerical Methods and PINNs for the Schrödinger equation

Mohsen Peykanro
Ehsan Pichka
Arshia Soleymani

Advisor:
Dr. Mahboubeh Molavi Arabshahi

July 13, 2026

Abstract

In this work, the numerical solution of the one-dimensional time-dependent Schrödinger equation with a harmonic potential is analyzed using the finite difference method (FDM) and physics-informed neural networks (PINNs). The complex-valued wave function is decomposed into its real and imaginary components, which are approximated simultaneously using a fully connected neural network. The governing differential equation, initial and boundary conditions, and reference solutions obtained using the FDM are incorporated into the PINN loss function. A two-stage optimization procedure employing the Adam and L-BFGS optimizers is used to train the neural network over the spatial interval $x \in [-5, 5]$ and the temporal interval $t \in [0, 1]$. The results demonstrate very good agreement between the PINN and FDM solutions. The relative L_2 errors of the predicted wave function and probability density are 1.5462×10^{-3} and 9.7108×10^{-4} , respectively, while the maximum absolute error of the probability density is 1.4054×10^{-3} . In addition, the network produces PDE residuals with root-mean-square values of approximately 10^{-3} and conserves the total probability with a maximum drift below 3×10^{-4} . Although the FDM achieves greater accuracy in terms of probability conservation, the PINN provides a continuous and mesh-free approximation across the entire space-time domain.

Keywords: Physics-informed neural networks; finite difference method; time-dependent Schrödinger equation; quantum harmonic oscillator.

1 Introduction

The Schrödinger equation is a partial differential equation in quantum mechanics that describes how the quantum state of a physical system change over time. This equation can handle the behavior of particles at the microscopic level. This equation have two forms that are based on times. The time-dependent Schrödinger equation describes the change of a system's wave function over time, while the time-independent version is used to determine stationary states and energy levels.

In most cases, you cant solve the equation so the exact analytical solutions are not available, especially for complex potentials or higher-dimensional systems. As a result, numerical methods play a crucial role in approximating solutions. The solution to the Schrödinger equation is called the wave function that provides information about the probability distribution of a particle's position and momentum. We mostly are going to survey Classical numerical approaches and Physics-Informed Neural Networks (PINNs) in this article.

The Classical numerical approaches such as the finite difference method (FDM) and finite element method (FEM) have been widely used for solving partial differential equations like Schrödinger equation. These methods rely on discretization of the domain and typically require careful handling of stability, convergence, and computational cost. On the other hand in recent years, PINNs have appeared as a modern approach for solving differential equations. PINNs merge the physical laws directly into the loss function of a neural network, so the solution will be approximated without discretization of the domain. This makes PINNs usefull for complex and high-dimensional problems.

Physics-informed neural computations have their theoretical underpinnings in the universal approximation theorem formulated by Hornik et al. [1]. The paper demonstrated that multilayer feedforward neural networks can approximate classes of continuous and measurable functions witharbitrary accuracy. This theoretical result is of great importancebecause it gave rise to the idea of representing unknown physical fields using the neural networks and, consequently, the solution of differential equations.

In the 1990s, efforts to combine neural networks with physical knowledge and differential equation modelling began. An early example was was proposed by Psychogios and Ungar [2] when they presented the hybrid neural network-first-principles approach for process modeling. Here, the neural network served as a tool to approximate the unknown process parameters,

whereas the known part of the system was modeled through physical relations like mass and energy balances. Even though the neural network was not used to approximate the solution of the differential equation in this case, this was one of the first attempts to incorporate prior physical knowledge into the neural network models.

Building on the universal approximation theorem proposed by Hornik et al. [1], the unknown solution of the PDE was approximated by a multi-layered feed-forward neural network as suggested by Dissanayake and Phan-Thien [3]. The parameters of the network were adjusted such that the objective function made up of the residual from the governing-equation and boundary conditions at certain collocation points was minimized. Given the similarity of the formulation of the objective function with those of current PINNs, the method suggested by Dissanayake and Phan-Thien might be regarded as an important precursor to modern physics-informed neural networks, even though physics-informed neural networks had not been invented yet.

Later in the decade, Lagaris et al. [4] proposed a framework for solving initial/boundary value problems defined by both ODEs and PDEs. Their framework included constructing the trial solutions through a known part satisfying the initial/boundary conditions and a variable part obtained by a neural network.

During the 2000s and early 2010s, improvements in computational power and numerical optimization led to the creation of highly developed techniques of solving differential equations using neural-network methods. As reviewed by Kumar and Yadav [5], a wide range of methods were proposed based on multilayer perceptron, radial-basis-function, hybrid and collocation methods of modeling ODEs and PDEs. The emergence of deep learning further increased the capacity of neural networks to perform complex computations, allowing researchers to train models consisting of multiple processing layers and therefore learn more complicated functions [6]. Moreover, the rise of software platforms, such as TensorFlow and Keras made building and differentiating deep neural networks easier, faster and more convenient [7, 8]. The first one provides a computation graph and automatic differentiation, making neural networks scalable across CPUs and GPUs, while the second allows defining and training the network architecture in a straightforward way.

In parallel with the above progress, researchers also investigated the use of deep learning for structured scientific problems. For instance, Kondor and Trivedi [9] presented a theory on how symmetry/equivariance can be considered within neural network frameworks, while Vlachas et al. [10] showed that LSTM networks can predict chaotic high-dimensional dynamical systems based solely on the input data. These works were not directly related to physics-informed neural networks, but they made significant contribution to the progress in deep-learning techniques for modelling physical and dynamical systems. Building upon earlier neural-network approaches for differential equations, advances in deep learning and automatic differentiation, and the growing integration of physical knowledge into machine learning, Raissi et al. [11] developed the modern PINN formulation for nonlinear PDEs.

Following the work of Raissi, Haghighat and Juanes [12] introduced SciANN, a TensorFlow- and Keras-based library developed for scientific computing and physics-informed deep learning. The applications of PINNs have expanded rapidly across a wide range of scientific and engineering disciplines. Researchers have successfully applied PINNs to nonlinear fluid dynamics, inverse parameter identification, computational finance, additive manufacturing, and other data-driven scientific computing problems [13, 14, 15, 16]. These studies demonstrate the versatility of PINNs in solving both forward and inverse PDE problems while effectively incorporating physical constraints into the learning process.

In this case, FDM and PINNs are used to find solutions for the one-dimensional time-dependent Schrödinger equation. The finite difference method is a traditional numerical approach where the differential operator in the equation is approximated at discrete points in a numerical grid. The physics-informed neural networks, however, use the loss function, where the PDE is part of the loss function and the conditions are added separately, while the neural

network attempts to minimize this loss function. FDM uses mesh discretization, whereas PINN can produce continuous solutions using automatic differentiation without a mesh. The results from both methods will be compared to the analytical solution in this paper.

2 Mathematical Formulation

2.1 Early-Stage Quantum Theory

The Schrödinger equation is a direct consequence of the discovery of quanta in the early 20th century. Its origins lie in the energy quanta hypothesis proposed by Max Planck in 1900 and the photon hypothesis by Albert Einstein in 1905, which established that energy and momentum are quantized.

$$E = h\nu = \hbar\omega \quad (1)$$

where E denotes energy, h is the Planck constant (6.626×10^{-34} J.s) and ν is frequency of light. so now we can define the reduced Planck constant as $\hbar = \frac{h}{2\pi}$ and $\omega = 2\pi\nu$ as angular frequency. Later on, Einstein in 1917 concluded that momentum of light quantum p is:

$$p = \frac{E}{c} = \frac{\hbar\omega}{c} = \hbar k \quad (2)$$

k is called wavenumber and is defined as $k = 2\pi/\lambda$, where λ is wavelength of light in vacuum.

In 1923, Arthur Compton conducted experiments investigating how incident X-ray beams were scattered by matter, such as graphite and copper. He discovered a systematic redshift (an increase in wavelength) in the scattered X-rays and they call it Compton effect. Moreover he found that the shift in wavelengths depended only on the scattering angle regardless of quality of material of a scatterer.

$$\Delta\lambda = \frac{h}{m_e c} (1 - \cos \theta) \quad (3)$$

where $\Delta\lambda$ means a Shift in wave length of scattered beam, m_e is a rest mass of an electron (mass that is independent of the motion of the particle) and θ is a scattering angle of the beam. The constant $\frac{h}{m_e c}$ is defined as the electron Compton wavelength (λ_e) approximately 2.426×10^{-12} meters.

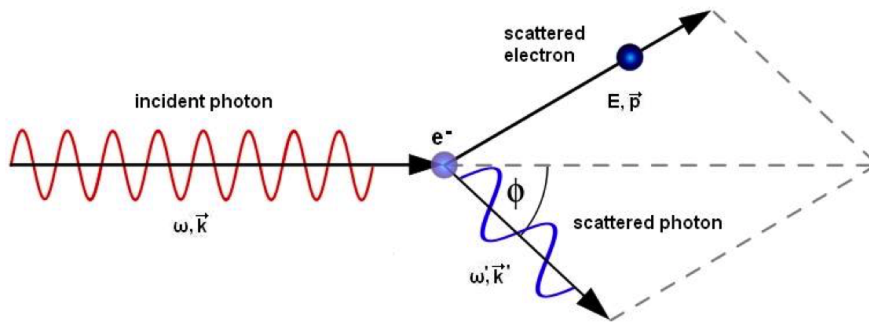


Figure 1: Compton effect: scattering of a photon on a resting electron

In 1924, Louis de Broglie Encouraged by the success of Einstein and Compton in proving that waves (light) could behave like particles and theorized that the reverse must also be true, in other word matter particles must possess a wave nature. De Broglie essentially reversed the relationship of (1) and (2) such that

$$\omega = \frac{E}{\hbar}, \quad k = \frac{p}{\hbar} \quad \text{or} \quad \lambda = \frac{h}{p} \quad (4)$$

where p equals $|p|$ and λ is a wavelength of a corpuscular beam (a directional stream of tiny particles). this is called de Broglie wavelength (matter wave). this equation implies that if we are able to determine the wavelength of the corpuscular beam experimentally, we can decide a magnitude of momentum accordingly.

For a matter wave, the phase velocity v_p is shown to be greater than c , while the group velocity v_g , which represents the propagation velocity of a wave packet (and thus the particle's velocity), is less than c . Their relationship is defined by $v_p v_g = c^2$. Therefore, a particle velocity must not exceed c .

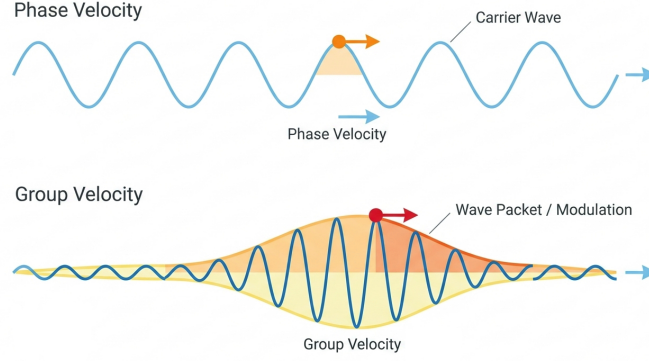


Figure 2: Phase velocity vs Group velocity

2.2 Time-Dependent Schrödinger Equation

The derivation starts with the standard wave equation

$$\nabla^2 \Psi = \frac{1}{v^2} \frac{\partial^2 \Psi}{\partial t^2} \quad (5)$$

where Ψ is an arbitrary function of a physical quantity relevant to propagation of a wave (shape of a wave), v is a phase velocity of wave, ∇^2 is called Laplacian and is defined as

$$\nabla^2 \equiv \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2} \quad (6)$$

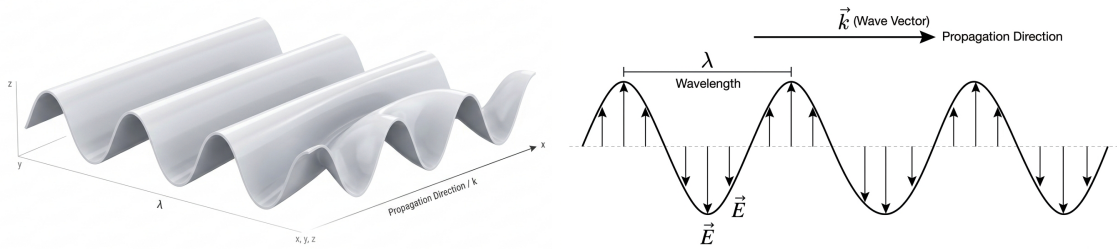


Figure 3: Plane waves in two and three dimensions

One of the special solutions for wave equation is called a plane wave and its expressed as

$$\Psi = \Psi_0 e^{i(k \cdot x - \omega t)} = \Psi_0 [\cos(k \cdot x - \omega t) + i \sin(k \cdot x - \omega t)] \quad (7)$$

where x denotes a position vector of a three-dimensional Cartesian coordinate. By applying de Broglie and Einstein's relations, this is converted into a matter wave representing a particle:

$$\Psi = \Psi_0 e^{i(\frac{p}{\hbar} \cdot x - \frac{E}{\hbar} t)}, \quad p = (e_1 \ e_2 \ e_3) \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix} \quad (8)$$

taking partial differentiation with respect to x we obtain

$$\frac{\partial \Psi}{\partial x} = \frac{i}{\hbar} p_x \Psi_0 e^{i(\frac{p}{\hbar}x - \frac{E}{\hbar}t)} = \frac{i}{\hbar} p_x \Psi \Rightarrow \frac{\hbar}{i} \frac{\partial \Psi}{\partial x} = p_x \Psi \quad (9)$$

Similarly we have

$$\frac{\hbar}{i} \frac{\partial \Psi}{\partial y} = p_y \Psi \quad \text{and} \quad \frac{\hbar}{i} \frac{\partial \Psi}{\partial z} = p_z \Psi \quad (10)$$

Taking partial differentiation once more

$$\frac{\partial^2 \Psi}{\partial x^2} = \left(\frac{i}{\hbar} p_x\right)^2 \Psi_0 e^{i(\frac{p}{\hbar}x - \frac{E}{\hbar}t)} = -\frac{1}{\hbar^2} p_x^2 \Psi \Rightarrow -\hbar^2 \frac{\partial^2 \Psi}{\partial x^2} = p_x^2 \Psi \quad (11)$$

Similarly we have

$$-\hbar^2 \frac{\partial^2 \Psi}{\partial y^2} = p_y^2 \Psi \quad \text{and} \quad -\hbar^2 \frac{\partial^2 \Psi}{\partial z^2} = p_z^2 \Psi \quad (12)$$

So if we summing both sides of these 3 equation and then dividing by $2m$ (m is the mass of a particle), we have

$$-\frac{\hbar^2}{2m} \nabla^2 \Psi - \frac{p^2}{2m} \Psi \quad (13)$$

Now lets get back to equation (8) and this time taking partial differentiation with respect to t , we obtain

$$\frac{\partial \Psi}{\partial t} = -\frac{i}{\hbar} E \Psi_0 e^{i(\frac{p}{\hbar}x - \frac{E}{\hbar}t)} = -\frac{i}{\hbar} E \Psi \Rightarrow i\hbar \frac{\partial}{\partial t} = E \Psi \quad (14)$$

If we subtract (13) and (14) we get

$$i\hbar \frac{\partial \Psi}{\partial t} + \frac{\hbar^2}{2m} \nabla^2 \Psi = (E - \frac{p^2}{2m}) \Psi \quad (15)$$

Comparing these equations we find out that momentum p becomes the operator $\frac{\hbar}{i} \nabla$ and energy E becomes the operator $i\hbar \frac{\partial}{\partial t}$. Now if we substituting these operators into the classical energy conservation relation (Total energy = Kinetic energy + Potential energy) we have

$$E = \frac{p^2}{2m} + V \quad (16)$$

So now lets rewrite the equation (15)

$$i\hbar \frac{\partial \Psi}{\partial t} + \frac{\hbar^2}{2m} \nabla^2 \Psi = V \Psi \Rightarrow \left(-\frac{\hbar^2}{2m} \nabla^2 + V\right) \Psi = i\hbar \frac{\partial \Psi}{\partial t} \quad (17)$$

This is the Time-Dependent Schrödinger Equation (TDSE), a fundamental equation of quantum mechanics.

If we define a following Hamiltonian operator H (the sum of two components, the kinetic energy and the potential energy of a quantum system) as

$$H \equiv -\frac{\hbar^2}{2m} \nabla^2 + V \quad (18)$$

Then we have a shorthand representation such that

$$H \Psi = i\hbar \frac{\partial \Psi}{\partial t} \quad (19)$$

Since we want to associate the wave function with a single particle, it is reasonable to require the probability to find the electron anywhere in space to be exactly equal to one. Since this may not be the case for all Ψ , we introduce a normalization condition:

$$\int_{-\infty}^{\infty} |\Psi(x, t)|^2 dx = 1 \quad (20)$$

The normalization thus imposes a condition on the wave function, the wave function must be a square integrable function.

2.3 Time-Independent Schrödinger Equation

In (17) Ψ is a function of x and t . Suppose that a potential V depends only upon x and separation of variables can be done with Ψ function ($\Psi(x, t) = \phi(x)\xi(t)$). By substituting these functions into the Time-Dependent Schrödinger Equation, we have

$$-\frac{\hbar^2}{2m}\nabla^2\phi(x)\xi(t) + V(x)\phi(x)\xi(t) = i\hbar\phi(x)\frac{\partial\xi(t)}{\partial t} \quad (21)$$

Now we will dividing the entire equation by $\phi(x)\xi(t)$

$$\frac{-\frac{\hbar^2}{2m}\nabla^2\phi(x)}{\phi(x)} + V(x) = \frac{i\hbar\frac{\partial\xi(t)}{\partial t}}{\xi(t)} \quad (22)$$

Since now the left hand side is only dependent on x and the right hand side only on t , both sides must be equal to a constant, which is defined as the total energy E , and we can now solve each side independently

$$H\phi(x) = E\phi(x) \quad (23)$$

$$\begin{aligned} i\hbar\frac{\partial\xi(t)}{\partial t} = E\xi(t) &\Rightarrow \frac{d\xi(t)}{\xi(t)} = \frac{E}{i\hbar} dt \Rightarrow \int \frac{d\xi}{\xi} = \frac{E}{i\hbar} \int dt \\ &\Rightarrow \ln \frac{\xi(t)}{\xi(0)} = \frac{Et}{i\hbar} \Rightarrow \xi(t) = \xi(0) \exp\left(-\frac{iEt}{\hbar}\right) \end{aligned} \quad (24)$$

The Time-Independent Schrödinger Equation (TISE)(23), is an eigenvalue problem where E represents the energy eigenvalues and $\phi(x)$ represents the eigenfunctions (or stationary states) of the Hamiltonian operator. After solving the problem, we get a solution

$$\Psi(x, t) = \phi(x) \exp\left(-\frac{iEt}{\hbar}\right) \quad (25)$$

3 Classical numerical methods for Schrödinger equation

Classical numerical methods for solving the Schrödinger equation work by turning a continuous problem into a form that can be handled by a computer. Some common methods include finite difference and finite element methods, which divide space into small pieces and approximate derivatives. Spectral methods, such as Fourier, Chebyshev, and B-spline methods, instead represent the solution using combinations of known functions. Other useful approaches include the split-step Fourier method and the Crank–Nicolson method, which are often used to simulate how a system changes over time. Monte Carlo methods can also be used, especially for more complex or higher-dimensional problems.

For time-dependent Schrödinger equations, methods like Crank–Nicolson, implicit–explicit schemes, and operator splitting are popular because they are stable and work well for step-by-step time evolution. On the other hand, time-independent problems are usually solved using matrix diagonalization, variational methods, or Monte Carlo techniques to find energy levels and corresponding wave functions.

3.1 Finite Difference Methods

Finite Difference Methods (FDM) are numerical techniques used to approximate solutions to differential equations by discretizing the equations in both time and space. The continuous derivatives in the equations are replaced with finite difference approximations, which transform the problem into a system of algebraic equations that can be solved computationally. FDM is widely used to solve both time-dependent and time-independent forms of the Schrödinger equation.

Discretizing the domain transforms the continuous space and time where the wave function $\Psi(x, t)$ exists into a finite set of specific points called a mesh or grid, because computers cannot handle an infinite continuum of points.

The spatial domain is divided into equally spaced grid points defined as

$$x_j = j\Delta x ; \quad j = 0, 1, 2, \dots, N \quad (26)$$

where Δx is the spatial step size and determines the resolution of the numerical approximation, with smaller values generally leading to higher accuracy at the expense of increased computational cost.

Similarly, the time domain is divided into discrete time levels. The time points are defined as

$$t_n = n\Delta t ; \quad n = 0, 1, 2, \dots \quad (27)$$

where Δt is the time step size and used to evolve the solution from one moment to the next.

After discretization, the continuous wave function $\Psi(x, t)$ is replaced by discrete values evaluated at the grid points. The value of the wave function at position x_j and time t_n is denoted by

$$\Psi_j^n \approx \Psi(x_j, t_n) \quad (28)$$

At any time step n the entire spatial state can be treated as a vector

$$U^n = \begin{pmatrix} \Psi_0^n \\ \Psi_1^n \\ \vdots \\ \Psi_N^n \end{pmatrix} \quad (29)$$

which represents the numerical state of the system at that particular time.

After discretizing the computational domain, the next step in the finite difference method is to approximate the derivatives appearing in the Schrödinger equation. Since numerical algorithms cannot directly evaluate continuous derivatives, these derivatives are replaced by algebraic expressions involving values of the wave function at neighboring grid points. This transformation converts the differential equation into a system of algebraic equations that can be solved using a computer.

The kinetic energy term in the Schrödinger equation contains the second spatial derivative of the wave function. A commonly used approximation is the second-order centered finite difference formula

$$\frac{\partial^2 \Psi}{\partial x^2} \approx \frac{\Psi_{j+1}^n - 2\Psi_j^n + \Psi_{j-1}^n}{\Delta x^2} \quad (30)$$

This approximation is second-order accurate, meaning that the truncation error is proportional to $O(\Delta x^2)$. As a result, reducing the grid spacing improves the accuracy of the numerical solution.

The time derivative of the wave function can also be approximated using finite differences. One of the simplest approximations is the forward difference scheme

$$\frac{\partial \Psi}{\partial t} \approx \frac{\Psi_j^{n+1} - \Psi_j^n}{\Delta t} \quad (31)$$

For improved accuracy, central difference approximations may be employed. In this case, the derivative is approximated by

$$\frac{\partial \Psi}{\partial t} \approx \frac{\Psi_j^{n+1} - \Psi_j^{n-1}}{2\Delta t} \quad (32)$$

After discretization and derivative approximation, the finite difference equations must be solved using an appropriate numerical strategy. Several implementation techniques have been developed depending on the type of Schrödinger equation being considered and the desired numerical properties, such as stability, efficiency, and accuracy.

- Boundary Management:

For a finite computational domain $\Omega = [a, b]$, Dirichlet boundary conditions impose homogeneous constraints on the wave function $\Psi(a, t) = \Psi(b, t) = 0$. This corresponds to an infinite potential barrier outside Ω , i.e.

$$V(x) = \begin{cases} V(x) & \text{if } x \in \Omega \\ \infty & \text{if } x \notin \Omega \end{cases} \quad (33)$$

To prevent waves from reflecting off the grid edges back into the simulation, an imaginary potential $\Im(V)$ is added to the Hamiltonian and modifies the continuity equation for probability density $\rho = \Psi\Psi^* = |\Psi|^2$. If $\Im(V)$ is negative, it acts as a sink, causing the probability density to decay as it enters the sponge region.

$$\frac{\partial \rho}{\partial t} + \nabla \cdot J = 2\rho\Im(V) \quad (34)$$

A typical smooth absorber is constructed using a hyperbolic tangent profile:

$$\Im(V) = -\frac{1}{2}V_0 \left\{ 2 + \tanh\left(\frac{(r - r_c)}{\delta}\right) - \tanh\left(\frac{r_c}{\delta}\right) \right\} \quad (35)$$

where V_0 is the depth of the well, r_c is the radius where the sponge starts, and δ is the transition width

For unbounded domains (infinite space), non-local boundary conditions allow waves to exit the domain as if the grid were infinite. These conditions are non-local in time, meaning the value at the boundary at time t depends on the history of the wave function at that point. For a potential V_- outside the boundary at $x = 0$, the boundary condition takes the form

$$\partial_x \Psi(0, t) = \sqrt{\frac{2}{\pi}} e^{-i\pi/4} e^{-iV_- t} \frac{d}{dt} \int_0^t \frac{\Psi(0, \tau) e^{iV_- \tau}}{\sqrt{t - \tau}} d\tau. \quad (36)$$

A homogeneous Dirichlet boundary condition is a type of essential boundary condition in which the value of the unknown function is prescribed to be zero on the boundary of the computational domain. It is mathematically defined as

$$\Psi(a, t) = \Psi(b, t) = 0; \quad \forall t \geq 0 \quad (37)$$

In the numerical solution of the Schrödinger equation, this condition restricts the wave function to vanish at the boundaries, preventing the particle probability density from existing outside the defined domain.

- The Crank-Nicolson Method:

The Crank-Nicolson method is an implicit finite difference scheme that combines the second-order central difference approximation in space with the trapezoidal rule (dividing the space into a row of trapezoids) in time, resulting in a numerical method that is second-order accurate, unconditionally stable, and preserves the unitary evolution of the quantum wave function.

Instead of evaluating the Hamiltonian only at t_n or t_{n+1} , the method uses the average

$$H\Psi \approx \frac{1}{2} (H\Psi^{n+1} + H\Psi^n) \quad (38)$$

Therefore, the discretized Schrödinger equation becomes

$$i\hbar \frac{\Psi_j^{n+1} - \Psi_j^n}{\Delta t} = \frac{1}{2} (H\Psi_j^{n+1} + H\Psi_j^n) \quad (39)$$

This symmetric treatment of the two time levels is the defining characteristic of the Crank-Nicolson method. Unlike other methods, Crank-Nicolson is unconditionally stable for any $r = \frac{\Delta t}{\Delta x^2}$.

3.2 Finite Element Methods

The Finite Element Method (FEM) is a numerical technique that approximates the solution by dividing the domain into a collection of small subdomains called finite elements, and finding an approximate solution using locally defined basis functions. Unlike the Finite Difference Method (FDM), which approximates derivatives directly at discrete grid points, FEM transforms the governing differential equation into its weak (variational) form before discretization. This formulation provides greater flexibility for handling complex geometries, irregular meshes and variable material properties.

Domain discretization is the first step. Let $\Omega \subset \mathbb{R}^d$ where $d = 1, 2, 3$ denote the domain. Such that

$$\bar{\Omega} = \bigcup_{e=1}^m \Omega_e \quad (40)$$

where Ω_e represents the e th finite element and m is the total number of elements. The interiors of any two distinct elements satisfy $\Omega_i \cap \Omega_j = \emptyset$ $i \neq j$ ensuring that the mesh completely covers the computational domain without overlap. In 1D, elements are intervals $[x_j, x_{j+1}]$, in 2D they are typically triangles or quadrilaterals and in 3D they are tetrahedra or hexahedra (bricks). Elements are interconnected at a finite number N of nodal points $\{x_k\}_{k=1}^N$ situated on their boundaries (and sometimes interiors)

The accuracy of the approximation depends strongly on the quality of the computational mesh. The mesh size is defined by

$$h = \max_{\Omega \in \tau_h} \text{diam}(\Omega); \quad \text{diam}(\Omega) = \max_{x, y \in \Omega} \|x - y\| \quad (41)$$

where $\text{diam}(\Omega)$ is the diameter of element Ω in partition τ_h . as the mesh is refined $h \rightarrow 0$ the finite element solution converges toward the exact solution. for convergence and numerical stability, the mesh is generally required to satisfy quasi-uniformity conditions, preventing excessively distorted elements. In practical applications involving irregular geometries, unstructured meshes are commonly used because they have better geometric flexibility than structured grids.

After choosing the mesh, a finite-dimensional approximation space is defined. For the Schrödinger equation, this space is typically chosen as $V_h \subset H^1(\Omega)$ (also known as Sobolev space) where $V_h = \text{span}\{N_1, N_2, \dots, N_N\}$ (also called shape functions) and each basis function is associated with a single mesh node. The shape functions satisfy the interpolation property:

$$N_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad (42)$$

This property guarantees that the nodal coefficients correspond directly to the values of the approximate solution at the mesh nodes. Furthermore, the basis functions satisfy the partition of unity:

$$\sum_{i=1}^N N_i(x) = 1; \quad \forall x \in \Omega \quad (43)$$

The continuous wave function is approximated by a finite-dimensional expansion over the chosen basis functions

$$\Psi(x, t) \approx \Psi_h(x, t) = \sum_{i=1}^N N_i(x) \Psi_i(t) \quad (44)$$

where $N_i(x)$ are the finite element basis functions and $\Psi_i(t)$ are the unknown time-dependent nodal coefficients representing the state of the system at node i . This discretization effectively transforms the TDSE into a system of ordinary differential equations in time which can be solved as a system of algebraic equations at each time step.

Instead of requiring the solution to satisfy the Schrödinger equation pointwise, the equation is reformulated as an integral identity over the computational domain. This approach reduces the differentiability requirements on the solution, making it possible to approximate the wave function using piecewise polynomial basis functions.

Let $\Omega \subset \mathbb{R}^d (d = 1, 2, 3)$ denote the computational domain with boundary $\partial\Omega$. The unknown wave function is sought in the Sobolev space

$$H_0^1(\Omega) = \{u \in H^1(\Omega) : u = 0 \text{ on } \partial\Omega\} \quad (45)$$

which consists of square-integrable functions whose first weak derivatives are also square-integrable and which satisfy Dirichlet boundary conditions. To derive the weak formulation let $\chi \in H_0^1(\Omega)$ be an arbitrary complex-valued test function. The core principle is that if the TDSE is satisfied at every point in the domain Ω , then the integral of the equation multiplied by any arbitrary test function must also be zero.

Consider the time-dependent Schrödinger equation

$$i\hbar \frac{\partial \Psi}{\partial t} = -\frac{\hbar^2}{2m} \nabla^2 \Psi + V\Psi + f \quad (46)$$

where $V(x)$ is the potential function and f denotes a source term. Multiplying the equation by the complex conjugate of the test function $\bar{\chi}$ and integrating over the computational domain yields

$$i\hbar \int_{\Omega} \frac{\partial \Psi}{\partial t} \bar{\chi} d\Omega = -\frac{\hbar^2}{2m} \int_{\Omega} \nabla^2 \Psi \bar{\chi} d\Omega + \int_{\Omega} V\Psi \bar{\chi} d\Omega + \int_{\Omega} f \bar{\chi} d\Omega \quad (47)$$

We define the inner product as

$$(u, v) = \int_{\Omega} u \bar{v} d\Omega \quad (48)$$

The (47) equation can be written compactly as

$$i\hbar(\Psi_t, \chi) = -\frac{\hbar^2}{2m}(\nabla^2 \Psi, \chi) + (V\Psi, \chi) + (f, \chi) \quad (49)$$

The Laplacian term contains second-order spatial derivatives so we will apply Green's first identity

$$\int_{\Omega} \nabla^2 \Psi \bar{\chi} d\Omega = - \int_{\Omega} \nabla \Psi \cdot \nabla \bar{\chi} d\Omega + \int_{\partial\Omega} \bar{\chi} \frac{\partial \Psi}{\partial n} d\Gamma \quad (50)$$

where $\frac{\partial \Psi}{\partial n} = \nabla \Psi \cdot \mathbf{n}$ denotes the outward normal derivative on the boundary. Since the test functions satisfy $\chi = 0$ on $\partial\Omega$, the boundary integral vanishes and resulting in

$$\int_{\Omega} \nabla^2 \Psi \bar{\chi} d\Omega = - \int_{\Omega} \nabla \Psi \cdot \nabla \bar{\chi} d\Omega \quad (51)$$

the second-order derivative acting on the wave function is replaced by first-order derivatives acting on both the trial and test functions. This reduction in derivative order is the defining feature of the weak formulation.

Substituting the results of integration by parts back into the integral statement yields the Weak Solution Formulation: Find $\Psi(\cdot, t) \in H_0^1(\Omega)$ such that

$$i\hbar(\Psi_t, \chi) = a(\Psi, \chi) + (f, \chi); \quad \forall \chi \in H_0^1(\Omega) \quad (52)$$

where $a(\Psi, \chi) = \frac{\hbar^2}{2m}(\nabla \Psi, \nabla \chi) + (V\Psi, \chi)$ is the bilinear form representing the Hamiltonian components. This variational equation forms the mathematical basis for the finite element discretization of the Schrödinger equation.

4 Physics-Informed Neural Networks

The conventional neural network is trained with the help of data only. If we provide some data samples to the network, the network tries to minimize the error between the predicted output values and the observed values. Though this methodology has produced very good results in many different scenarios, it needs plenty of data and does not ensure that the solution that is predicted by the model complies with the physical laws.

On the other hand, Physics-Informed Neural Networks (PINNs) combine the concept of a neural network with the physical laws governing the system. Unlike a standard neural network, which is trained using data only, a PINN is constrained by physical laws represented by a Partial Differential Equation (PDE). Here, the neural network is responsible for approximating the unknown solution ($u(t, x)$), whereas automatic differentiation is used to calculate the necessary derivatives in the PDE. For a general PDE of the form

$$u_t + \mathcal{N}[u] = 0 \quad (53)$$

the neural network predicts the solution ($u(t, x)$). Using automatic differentiation, the derivatives (u_t), (u_x), (u_{xx}), and higher-order terms can be computed exactly with respect to the network inputs. These derivatives are then substituted into the governing equation to form a residual function

$$f(t, x) = u_t + \mathcal{N}[u]. \quad (54)$$

In case the neural network adheres exactly to the physical law, then the residual ($f(t, x)$) is supposed to take the value of zero throughout the whole domain. Hence, there will be two components in the training process of a PINN. One is related to the data error while the other component relates to physics, which serves as an additional constraint.

A PINN may therefore be regarded as a neural network that makes use of both observation data as well as the principles of physics. The incorporation of such physical knowledge provides additional information for the network, which works as a regularization technique.

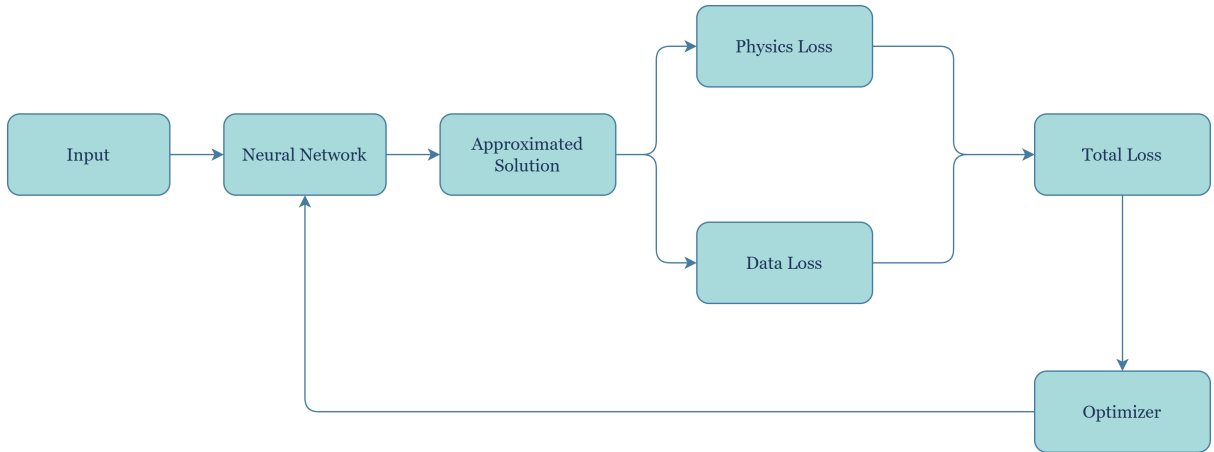


Figure 4: PINN Framework

The architecture of a Physics-Informed Neural Network is typically implemented based on a fully connected feed-forward neural network, though in many cases they try many other architecture for NN. In this structure, the input layer receives the independent variables of the problem. For a time-dependent one-dimensional PDE, these inputs are usually the spatial coordinate (x) and the time variable (t). Therefore, each input point can be shown as (x, t) . The role of the neural network is mapping this input point to the corresponding approximation of the unknown solution. In other words, the network represents the function

$$u_\theta(x, t) \approx u(x, t) \quad (55)$$

where (θ) denotes all trainable parameters of the neural network, including weights and biases.

In the traditional neural network, the output of the architecture is checked only with available data. However, the output from the architecture in a PINN is further taken to the physics-based part of the algorithm. This means that after the network predicts $u_\theta(x, t)$, automatic differentiation is applied to compute the derivatives required in the PDE, such as

$$\frac{\partial u_\theta}{\partial t}, \frac{\partial u_\theta}{\partial x}, \frac{\partial^2 u_\theta}{\partial x^2}. \quad (56)$$

To evaluate how well the governing equation is satisfied, PINNs introduce a set of collocation points inside the computational domain. These points are sampled throughout the domain and are not necessarily associated with any observed data. Let the collocation points be denoted by

$$(x_f^i, t_f^i)_{i=1}^{N_f}$$

where (N_f) represents the number of collocation points. At each of these points, the residual $(f_\theta(x, t))$ is evaluated. If the residual is large at a particular location, it indicates that the neural network prediction violates the governing equation at that point. Consequently, the network parameters are adjusted to reduce this error. One important advantage of this approach is that PINNs are mesh-free methods and therefore do not require the computational mesh used by traditional numerical techniques such as the Finite Element Method or Finite Difference Method [11] [17].

The learning process of a PINN is controlled through a loss function that combines information from both the available data and the governing physics. In the most general case, the total loss function consists of four components:

$$\mathcal{L}_{data} + \mathcal{L}_{physics} + \mathcal{L}_{boundary} + \mathcal{L}_{initial}$$

The data loss measures the difference between the neural network predictions and the available observations. The physics loss evaluates how well the governing differential equation is satisfied at the collocation points. The boundary loss enforces the prescribed boundary conditions, while the initial loss ensures that the initial conditions are satisfied. Together, these terms guide the network toward a physically meaningful solution. More specifically, the physics loss is often defined as the mean squared value of the PDE residual evaluated at all collocation points:

$$\frac{1}{N_f} \sum_{i=1}^{N_f} |f_\theta(x_f^i, t_f^i)|^2 \quad (57)$$

Minimizing this term forces the neural network to satisfy the governing equation throughout the computational domain. Similarly, if observational data are available, the data loss can be written as

$$\frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(x_u^i, t_u^i) - u^i|^2, \quad (58)$$

where (u^i) denotes the observed value of the solution at the point $((x_u^i, t_u^i))$.

After the total loss function is constructed, an optimization algorithm such as Adam or L-BFGS is used to update the trainable parameters (θ) . During each training iteration, the gradients of the loss function with respect to the network parameters are computed using back-propagation. The optimization process continues until the total loss reaches a sufficiently small value, indicating that the neural network simultaneously fits the available data and satisfies the governing physical laws.

The Schrödinger equation is the fundamental equation of quantum mechanics and describes the evolution of a quantum system through its wave function. In its time-dependent form, the equation is written as

$$i\hbar \frac{\partial \psi}{\partial t} = \hat{H}\psi, \quad (59)$$

where $\psi(x, t)$ denotes the wave function, $\hbar$ is the reduced Planck constant, and $\hat{H}$ is the Hamiltonian operator. The solution of the Schrödinger equation contains complete information about the physical system and can be used to determine quantities such as probability distributions and energy levels.

The Schrödinger equation plays a central role in many scientific fields, including quantum mechanics, quantum chemistry, atomic physics, condensed matter physics, and quantum computing. Because analytical solutions are available only for a limited number of simple systems, numerical methods are often required to approximate its solutions.

In recent years, Physics-Informed Neural Networks (PINNs) have emerged as a promising alternative to traditional numerical approaches. By embedding the governing differential equation directly into the loss function, PINNs can learn solutions that satisfy both observational data and the underlying physical laws. This capability makes PINNs particularly attractive for solving Schrödinger-type equations, where accurate approximations of wave functions are required [18].

4.1 PINN Formulation for the Schrödinger Equation

In this project, we consider the one-dimensional time-dependent Schrödinger equation with a harmonic potential

$$i\frac{\partial \psi}{\partial t} = -\frac{1}{2}\frac{\partial^2 \psi}{\partial x^2} + \frac{1}{2}x^2\psi, \quad (60)$$

where $\psi(x, t)$ denotes the complex-valued wave function. The term

$$-\frac{1}{2}\frac{\partial^2 \psi}{\partial x^2} \quad (61)$$

represents the kinetic energy contribution, while

$$\frac{1}{2}x^2\psi \quad (62)$$

corresponds to the harmonic oscillator potential. This equation is one of the most important models in quantum mechanics and is frequently used as a benchmark problem for numerical and machine learning methods.

To solve this problem using PINNs, the neural network receives the spatial coordinate x and time t as inputs and approximates the wave function $\psi(x, t)$. Automatic differentiation is then employed to compute the derivatives required by the governing equation, and the residual of the Schrödinger equation is incorporated into the loss function. The network parameters are optimized such that the predicted solution satisfies both the differential equation and the prescribed initial and boundary conditions.

To obtain a unique solution of the Schrödinger equation, appropriate initial and boundary conditions must be specified. In this work, the spatial domain is defined as

$$x \in [-5, 5] \quad (63)$$

while the temporal domain is chosen as

$$t \in [0, 1] \quad (64)$$

The initial condition is selected as a Gaussian wave packet,

$$\psi(x, 0) = e^{-x^2} \quad (65)$$

which is a commonly used initial state in quantum mechanics due to its smoothness and localized nature. Since the wave function is represented in terms of its real and imaginary components,

$$\psi(x, t) = u(x, t) + iv(x, t) \quad (66)$$

the initial condition can be written as

$$u(x, 0) = e^{-x^2} \quad (67)$$

$$v(x, 0) = 0 \quad (68)$$

These conditions are enforced during training through the initial-condition loss term, which penalizes discrepancies between the neural network predictions and the prescribed initial values.

Furthermore, homogeneous Dirichlet boundary conditions are imposed at the boundaries of the spatial domain. Specifically,

$$u(-5, t) = u(5, t) = 0 \quad (69)$$

and

$$v(-5, t) = v(5, t) = 0 \quad (70)$$

These boundary conditions ensure that the wave function remains negligible at the edges of the computational domain, which is consistent with the localized nature of the Gaussian initial state.

Within the PINN framework, both the initial and boundary conditions are incorporated directly into the loss function. During training, the neural network is required not only to satisfy the governing Schrödinger equation but also to reproduce the prescribed initial and boundary conditions. This enables the network to learn a physically consistent approximation of the wave function throughout the spatio-temporal domain.

To solve the time-dependent

Schrödinger equation using Physics-Informed Neural Networks (PINNs), the complex-valued wave function is first decomposed into its real and imaginary components. Thus, the solution is represented as

$$\psi(x, t) = u(x, t) + i v(x, t) \quad (71)$$

where $u(x, t)$ and $v(x, t)$ denote the real and imaginary parts of the wave function, respectively.

In the PINN framework, a fully connected neural network is employed to approximate these two functions simultaneously. The inputs of the network are the spatial coordinate x and the time variable t , while the outputs correspond to the approximations $u_\theta(x, t)$ and $v_\theta(x, t)$, where θ denotes the trainable parameters of the neural network.

The governing equation considered in this work is

$$i \frac{\partial \psi}{\partial t} = -\frac{1}{2} \frac{\partial^2 \psi}{\partial x^2} + \frac{1}{2} x^2 \psi \quad (72)$$

Substituting

$$\psi = u + i v \quad (73)$$

into the Schrödinger equation and separating the real and imaginary parts yields the coupled system

$$u_t = -\frac{1}{2}v_{xx} + V(x)v \quad (74)$$

$$v_t = \frac{1}{2}u_{xx} - V(x)u \quad (75)$$

where

$$V(x) = \frac{1}{2}x^2 \quad (76)$$

is the harmonic potential.

Using automatic differentiation, the neural network computes the required derivatives u_t , v_t , u_{xx} , and v_{xx} . These derivatives are then used to construct the physics residuals

$$f_u = u_t + \frac{1}{2}v_{xx} - V(x)v \quad (77)$$

$$f_v = v_t - \frac{1}{2}u_{xx} + V(x)u \quad (78)$$

If the neural network exactly satisfies the Schrödinger equation, both residual functions vanish throughout the computational domain. Therefore, the objective of the PINN is to minimize these residuals while simultaneously satisfying the prescribed initial and boundary conditions.

To enforce the governing equation, a set of collocation points is sampled inside the spatio-temporal domain. At these points, the residuals f_u and f_v are evaluated and incorporated into the loss function. The resulting physics loss is given by

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} (|f_u(x_i, t_i)|^2 + |f_v(x_i, t_i)|^2) \quad (79)$$

where N_f denotes the number of collocation points.

The total loss function combines the physics loss with the losses associated with the initial conditions, boundary conditions, and available reference data. Therefore,

$$\mathcal{L} = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{Data}}\mathcal{L}_{\text{Data}} \quad (80)$$

where the coefficients λ_{PDE} , λ_{IC} , λ_{BC} , and λ_{Data} control the contribution of each component during training.

5 Results and Discussion

The performance of the proposed physics-informed neural network was evaluated from four complementary perspectives: training convergence, agreement with the finite-difference reference solution, satisfaction of the governing differential equation, and conservation of the wave-function norm. The results demonstrate that the trained network accurately reproduced the spatio-temporal evolution of the wave function while maintaining small equation residuals and satisfactory physical consistency throughout the computational domain.

The optimization process produced a substantial reduction in all components of the loss function. During the first training stage, the Adam optimizer rapidly moved the network

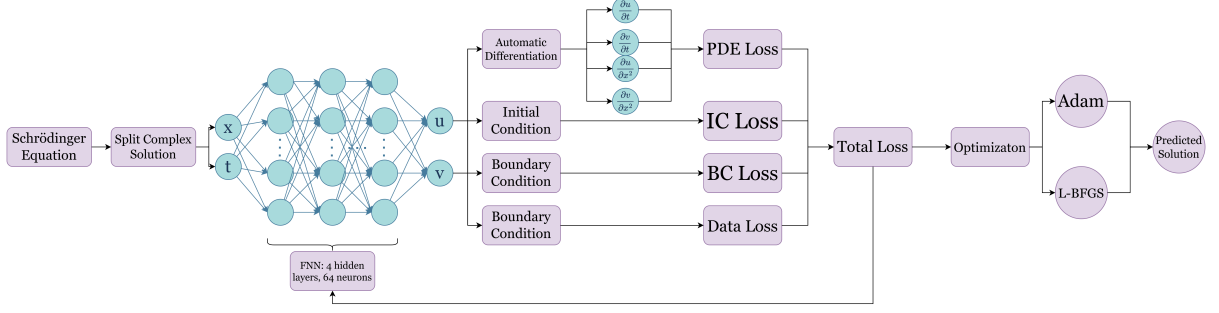


Figure 5: Diagram of solving Schrodinger with PINN

parameters toward a suitable approximation of the solution. The subsequent L-BFGS stage further refined the parameters and reduced the remaining error. This two-stage strategy was particularly effective because Adam provided a stable initial approximation, while L-BFGS improved the final satisfaction of the physical and observational constraints.

The total loss was defined as the sum of the PDE, boundary-condition, initial-condition, and reference-data contributions:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{BC}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{data}}. \quad (81)$$

All four grouped loss components decreased by several orders of magnitude during training. The boundary-condition loss reached the smallest final value, indicating that the homogeneous boundary conditions were satisfied with high accuracy. The PDE loss remained the largest component at the end of optimization because the governing equation had to be enforced throughout the entire space-time domain rather than only along the boundaries or initial line. The total training and testing losses followed nearly identical convergence patterns, indicating consistent performance across the sampled training and testing points. This agreement, however, should be interpreted as consistency on the sampled sets rather than definitive proof that overfitting is impossible.

The complete convergence history, including the transition from Adam to L-BFGS, is illustrated in Figure 6.

At the end of training, the total loss reached

$$\mathcal{L}_{\text{total}} = 2.4256 \times 10^{-6}. \quad (82)$$

The combined PDE loss was 1.8756×10^{-6} , accounting for approximately 77.3% of the remaining total loss. By comparison, the boundary, initial-condition, and reference-data losses were 3.0547×10^{-8} , 2.3427×10^{-7} , and 2.8515×10^{-7} , respectively. These values indicate that the network satisfied the auxiliary constraints more accurately than the differential equation itself, which is expected because the PDE residual must be minimized over the full interior of the computational domain.

The real and imaginary PDE losses were similar, suggesting balanced enforcement of the two coupled equations. A greater difference was observed between the initial-condition losses of the two components, with the imaginary-component loss being larger than the real-component loss. Nevertheless, both values remained small and did not prevent the network from achieving high global accuracy. The final individual and grouped loss values are summarized in Table 1.

The accuracy of the trained network was further evaluated by comparing its predicted probability density with the reference solution generated using the finite-difference method. The probability density was calculated from the real and imaginary components as

$$\rho(x, t) = |\psi(x, t)|^2 = u(x, t)^2 + v(x, t)^2. \quad (83)$$

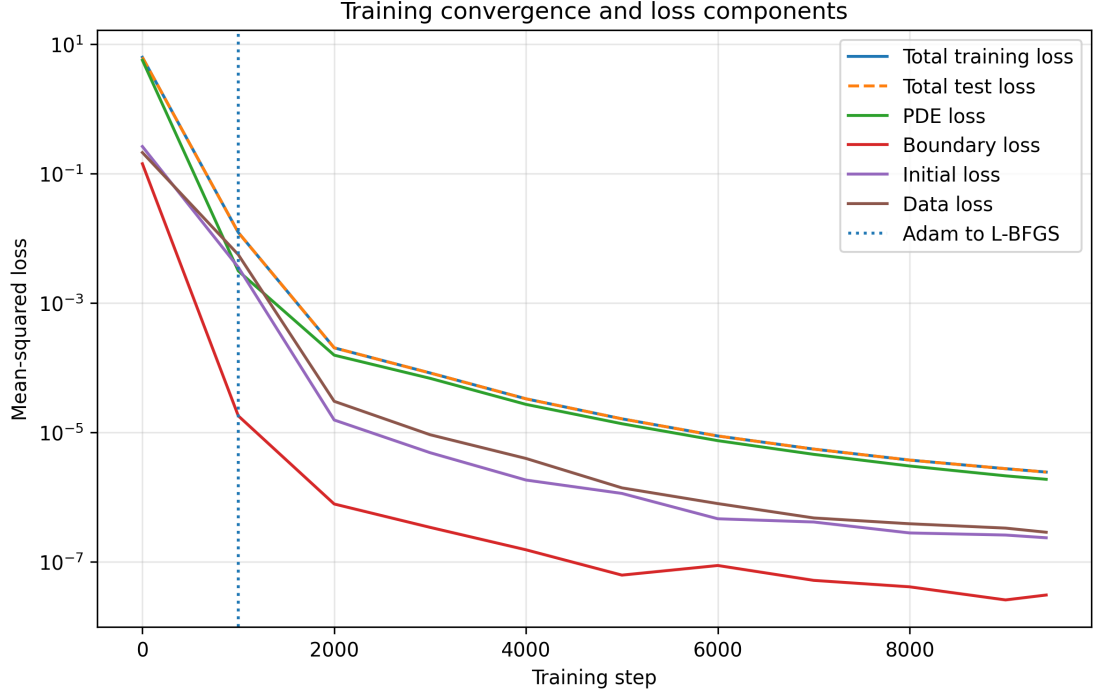


Figure 6: Convergence history of the total training and testing losses and the individual PINN loss components. The vertical dotted line indicates the transition from Adam to L-BFGS.

Table 1: Final individual and grouped PINN loss components.

Loss component	Final value
Real PDE residual loss	9.4813×10^{-7}
Imaginary PDE residual loss	9.2750×10^{-7}
Boundary loss for u	1.7885×10^{-8}
Boundary loss for v	1.2662×10^{-8}
Initial-condition loss for u	2.9938×10^{-8}
Initial-condition loss for v	2.0433×10^{-7}
Reference-data loss for u	1.4485×10^{-7}
Reference-data loss for v	1.4029×10^{-7}
Combined PDE loss	1.8756×10^{-6}
Combined boundary loss	3.0547×10^{-8}
Combined initial-condition loss	2.3427×10^{-7}
Combined reference-data loss	2.8515×10^{-7}
Total training loss	2.4256×10^{-6}

Close agreement was obtained between the PINN and FDM probability-density fields throughout the entire computational domain. Both methods produced the same overall spatio-temporal structure, including the central concentration of the wave packet and its gradual broadening over the simulated time interval. No visible displacement or distortion of the principal density structure was observed in the PINN prediction.

The pointwise error was calculated using

$$E_\rho(x, t) = |\rho_{\text{PINN}}(x, t) - \rho_{\text{FDM}}(x, t)|. \quad (84)$$

The largest errors were concentrated near the central region, where the probability density was greatest and changed most rapidly in space and time. Much smaller errors occurred toward the boundaries because the wave-function magnitude approached zero in those regions. This spatial distribution is reasonable because reproducing the height, width, and temporal evolution of the central wave packet is more difficult than approximating nearly zero values far from its centre.

The reference field, PINN prediction, and corresponding absolute-error distribution are compared in Figure 7. An identical colour scale was used for the reference and predicted density fields so that their visual agreement could be assessed directly.

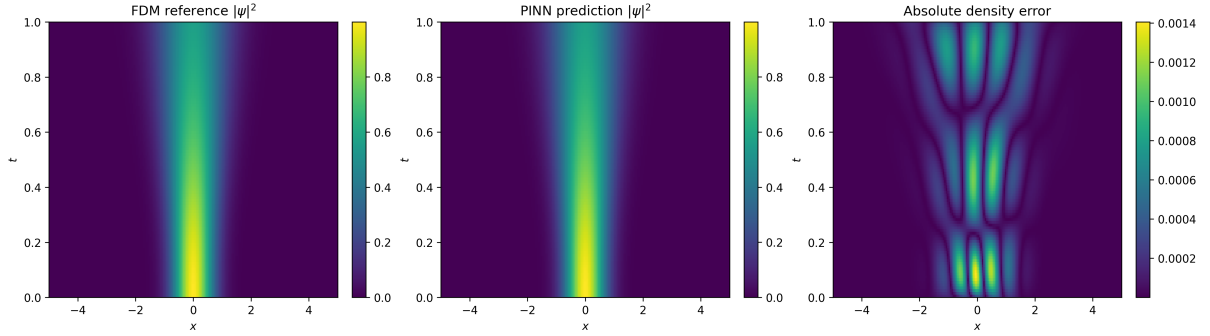


Figure 7: Comparison of the FDM reference probability density, the PINN prediction, and the pointwise absolute density error throughout the space-time domain.

The global relative L_2 error was calculated as

$$\varepsilon_q = \frac{\|q_{\text{PINN}} - q_{\text{FDM}}\|_2}{\|q_{\text{FDM}}\|_2}, \quad (85)$$

where q represents u , v , ψ , or ρ . The resulting relative error of the complete complex wave function was

$$\varepsilon_\psi = 1.5462 \times 10^{-3}, \quad (86)$$

corresponding to approximately 0.155%. The relative error of the probability density was even smaller:

$$\varepsilon_\rho = 9.7108 \times 10^{-4}, \quad (87)$$

which corresponds to approximately 0.097%. Therefore, the global density error remained below one tenth of one percent.

The real component had a relative error of 1.1709×10^{-3} , while the imaginary component had a larger relative error of 3.2844×10^{-3} . This difference does not necessarily indicate a substantial weakness in the prediction of v , because a component-wise relative error also depends on the norm of the corresponding reference component. The error of the complete complex wave function is consequently a more representative measure of the overall solution quality.

The mean absolute density error and root-mean-square density error were 1.2451×10^{-4} and 2.5485×10^{-4} , respectively. The maximum pointwise density error was 1.4054×10^{-3} . The fact that the mean and RMS errors were considerably smaller than the maximum error indicates that the largest discrepancies were localized rather than distributed throughout the domain. The complete accuracy measures are reported in Table 2.

Table 2: Quantitative errors between the PINN and FDM solutions.

Error measure	Value
Relative L_2 error of u	1.1709×10^{-3}
Relative L_2 error of v	3.2844×10^{-3}
Relative L_2 error of ψ	1.5462×10^{-3}
Relative L_2 error of $ \psi ^2$	9.7108×10^{-4}
Density mean absolute error	1.2451×10^{-4}
Density root-mean-square error	2.5485×10^{-4}
Maximum absolute density error	1.4054×10^{-3}

The full-field comparison was complemented by examining the probability-density profiles at representative time instants. This comparison provides a more detailed view of whether the network reproduced the local peak, width, symmetry, and spatial decay of the wave packet.

At $t = 0$, the predicted density closely matched the prescribed initial profile. As time progressed, the central peak gradually decreased while the distribution became wider. This behaviour indicates a redistribution of the wave-function density over a broader spatial region. The reduction in peak height does not represent a loss of norm because the area under the density profile remained approximately constant.

The PINN and FDM curves overlapped closely at

$$t = 0, \quad 0.25, \quad 0.50, \quad 0.75, \quad 1.00.$$

The agreement was maintained not only near the maximum density but also along the tails of the distribution. Consequently, the network accurately reproduced both the amplitude and the spatial broadening of the solution over the complete temporal interval. These selected-time comparisons are presented in Figure 8.

The near-complete overlap of the curves is consistent with the small global density errors reported previously. More importantly, the line profiles confirm that the strong agreement observed in the heatmaps was not caused by colour-scale smoothing. The predicted and reference solutions remained close when compared directly as one-dimensional functions of the spatial coordinate.

Satisfaction of the governing equation was investigated by evaluating the two coupled residuals produced by the real-valued formulation of the Schrödinger equation:

$$f_u(x, t) = u_t + \frac{1}{2}v_{xx} - V(x)v, \quad (88)$$

$$f_v(x, t) = v_t - \frac{1}{2}u_{xx} + V(x)u. \quad (89)$$

For an exact solution, both residuals would be zero throughout the computational domain. The trained PINN produced residuals that remained small over most of the space-time region. Their magnitudes generally occurred around 10^{-3} or below, although several localized regions reached values of several 10^{-3} .

The residual distributions contained curved low-value bands where the signed residuals passed through zero. These structures do not represent discontinuities in the wave function.

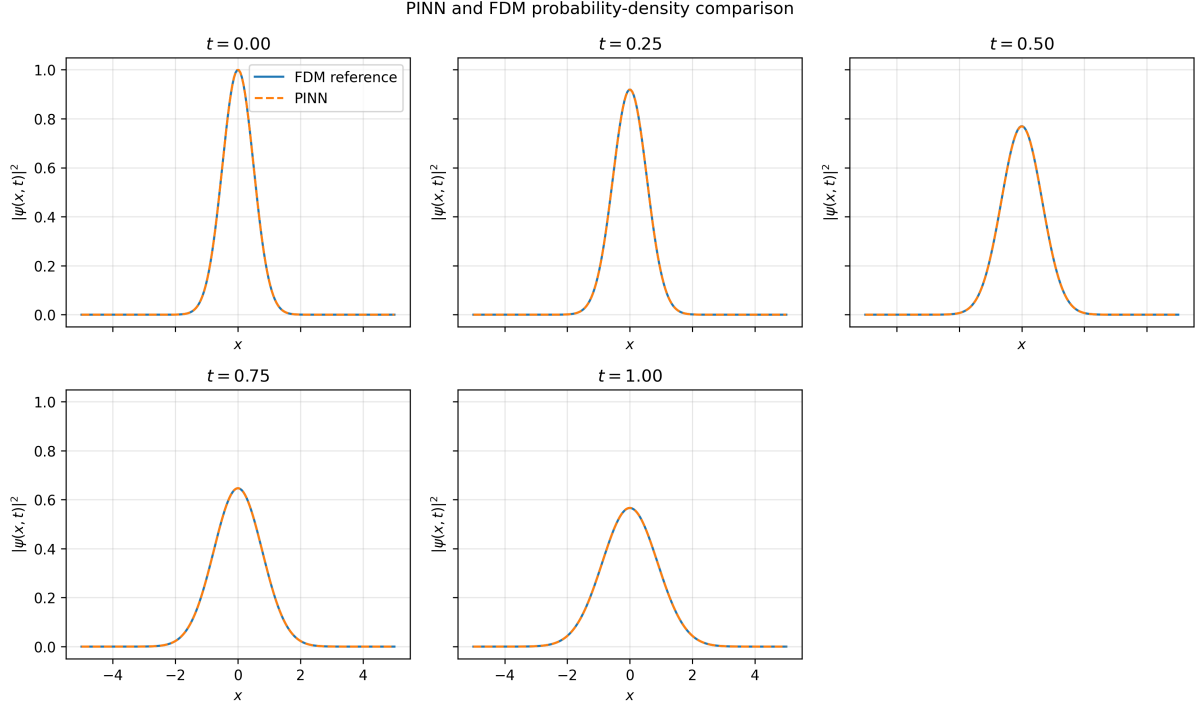


Figure 8: Comparison of the PINN and FDM probability-density profiles at selected time instants.

They result from plotting the absolute residual magnitude, which becomes very small wherever the signed residual changes sign.

A logarithmic colour scale was used because the residual values covered several orders of magnitude. The resulting distributions of $|f_u|$ and $|f_v|$ are shown in Figure 9.

The mean absolute residuals were 7.5442×10^{-4} and 7.5263×10^{-4} for the real and imaginary equations, respectively. Their RMS values were approximately 1.0×10^{-3} , whereas the maximum residuals were 7.4639×10^{-3} and 6.7442×10^{-3} .

These values are consistent with the final PDE loss. Since the PDE loss is based on squared residuals, an RMS residual of approximately 10^{-3} produces a squared contribution of approximately 10^{-6} . The numerical agreement between the residual statistics and the loss history confirms that the two diagnostics are internally consistent.

The maximum values were approximately seven times greater than the corresponding RMS residuals, showing that the remaining PDE mismatch was concentrated in relatively small regions rather than spread uniformly across the domain. No extensive regions of persistently large residual were observed, supporting the conclusion that the PINN satisfied the governing physics globally. The residual statistics are summarized in Table 3.

Table 3: Statistical measures of the real and imaginary PDE residuals.

Statistic	f_u	f_v
Mean absolute residual	7.5442×10^{-4}	7.5263×10^{-4}
RMS residual	1.0379×10^{-3}	1.0209×10^{-3}
Maximum absolute residual	7.4639×10^{-3}	6.7442×10^{-3}

An additional assessment of physical consistency was performed by calculating the integrated probability density:

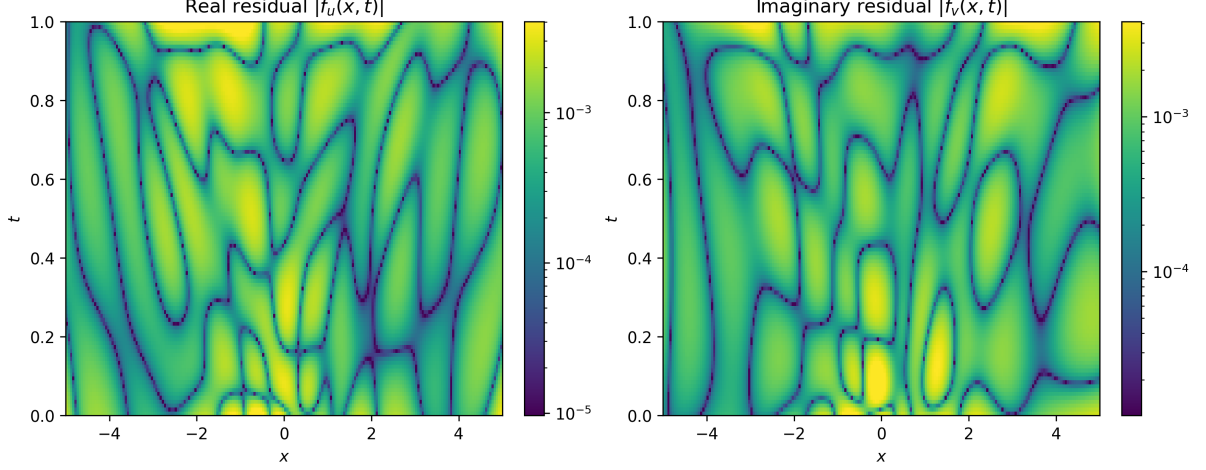


Figure 9: Absolute values of the real and imaginary Schrödinger equation residuals over the computational domain. A logarithmic colour scale is used to display the residual magnitudes.

$$N(t) = \int_{-5}^5 |\psi(x, t)|^2 dx. \quad (90)$$

For Schrödinger evolution under a real potential, this quantity should remain constant in time. The finite-difference reference solution preserved the norm almost exactly, producing a horizontal curve over the full temporal interval. This behaviour is expected because the matrix-exponential time evolution used by the reference solver preserves the discrete wave-function norm to numerical precision.

The PINN result was also nearly constant but displayed small oscillations around the reference level. These fluctuations arise because the neural network satisfies the governing equation and constraints approximately rather than exactly. Importantly, no systematic increasing or decreasing trend was observed, indicating that the model did not experience substantial accumulated norm drift.

The initial norm was approximately 1.2533, rather than unity. Therefore, the plotted quantity should be described as the wave-function norm rather than total probability. This value indicates that the initial wave function used in the experiment was not normalized to one. Although the lack of normalization does not affect the comparison between the PINN and FDM solutions, normalization would be preferable for a standard quantum-mechanical interpretation.

The temporal evolution of the integrated density is displayed in Figure 10.

The maximum absolute PINN norm deviation was

$$2.9667 \times 10^{-4}, \quad (91)$$

corresponding to a maximum relative deviation of

$$2.3671 \times 10^{-4} \approx 0.0237\%. \quad (92)$$

The finite-difference norm deviation was approximately 7.4303×10^{-13} in relative terms, which is effectively at machine precision. The maximum difference between the PINN and FDM norms was 2.8269×10^{-4} . These results indicate that the PINN preserved the norm with high accuracy despite not imposing exact norm conservation as a separate training constraint. The corresponding conservation measures are provided in Table 4.

The combined numerical evidence demonstrates that the proposed PINN accurately approximated the solution of the time-dependent Schrödinger equation over the considered domain.

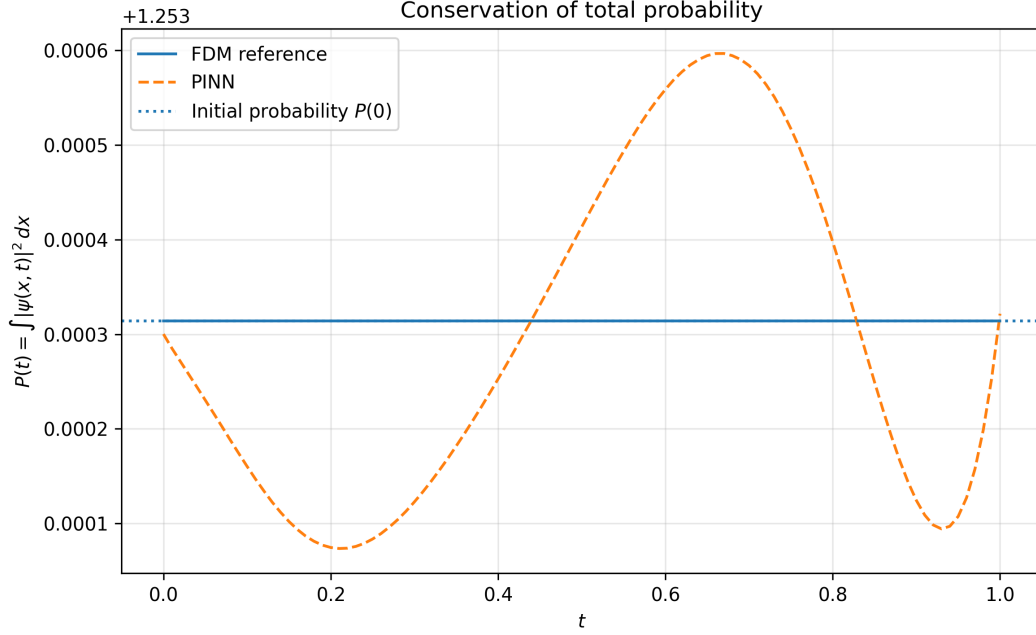


Figure 10: Temporal evolution of the wave-function norm obtained from the PINN and FDM solutions.

Table 4: Wave-function norm conservation for the PINN and FDM solutions.

Norm measure	Value
Initial PINN norm	1.25330016
Minimum PINN norm	1.25307344
Maximum PINN norm	1.25359683
Maximum absolute PINN norm deviation	2.9667×10^{-4}
Maximum relative PINN norm deviation	2.3671×10^{-4}
Initial FDM norm	1.25331414
Maximum absolute FDM norm deviation	9.3126×10^{-13}
Maximum relative FDM norm deviation	7.4303×10^{-13}
Maximum PINN–FDM norm difference	2.8269×10^{-4}

Stable convergence was achieved through the hybrid Adam and L-BFGS optimization procedure, resulting in a final loss of 2.4256×10^{-6} . The boundary and initial conditions were satisfied accurately, while the PDE residual formed the largest remaining contribution to the loss.

Strong agreement was obtained between the PINN and finite-difference solutions at both the field and profile levels. The relative L_2 error of the complete wave function was approximately 0.155%, while the corresponding density error was approximately 0.097%. The maximum point-wise density error remained limited to 1.4054×10^{-3} , and the much smaller MAE and RMSE values showed that this maximum discrepancy occurred only in localized regions.

The residual analysis confirmed that the small prediction error was accompanied by satisfactory enforcement of the governing equation. Both real and imaginary residuals had RMS magnitudes of approximately 10^{-3} , which was consistent with their squared loss contributions of approximately 10^{-6} . The close similarity between the residual statistics of the two equations further indicated that neither component of the coupled system was substantially less well represented.

Physical consistency was also maintained throughout the simulation. The PINN preserved the wave-function norm with a maximum relative deviation of only 0.0237%. Although this level of conservation was less exact than that of the finite-difference reference method, it remained sufficiently small to confirm that the predicted evolution did not suffer from significant artificial gain or loss of norm.

Several limitations should nevertheless be acknowledged. The present model was evaluated for one potential, one temporal interval, and one network configuration. In addition, reference-data constraints were included during training, meaning that the proposed model represents a data-assisted PINN rather than a completely data-free formulation. Further experiments should therefore examine the performance of the model without reference data, with different numbers of collocation points, and with alternative network depths and widths. Longer temporal intervals and more complicated potentials would also provide a stronger assessment of generalization.

The initial wave function should be normalized in future experiments so that the conserved integral equals one and can be interpreted directly as total probability. An ablation study comparing Adam-only training with the hybrid Adam–L-BFGS procedure would additionally clarify the contribution of the second optimization stage. Despite these limitations, the current results provide strong evidence that the proposed PINN formulation can accurately reproduce the dynamics of the investigated Schrödinger system while satisfying the governing equation and preserving its principal integral property.

6 Conclusion

In this study, a physics-informed neural network was developed to solve the one-dimensional time-dependent Schrödinger equation with a harmonic potential, and its predictions were compared with a finite-difference reference solution. The numerical results showed a close agreement between the two approaches over the entire space-time domain. The PINN reproduced the evolution of the wave function and probability density with small relative errors, while the low PDE residuals confirmed that the governing equation was satisfied accurately. The model also preserved the wave-function norm with only a small deviation, although the finite-difference method achieved conservation to near machine precision.

Overall, the results demonstrate that the proposed data-assisted PINN is a reliable approach for approximating the time-dependent Schrödinger equation. While the finite-difference method remains highly accurate for the structured one-dimensional problem considered here, the PINN provides a continuous and flexible approximation that can be evaluated at arbitrary points without constructing a new spatial grid. The ideas presented in this work may therefore support the further development of physics-informed models for more complex quantum systems, including problems with time-dependent potentials, longer simulation intervals, and higher-dimensional

domains. Future studies should also examine purely physics-informed formulations without reference data and investigate the influence of network architecture, collocation-point distribution, and optimization strategy on the accuracy and stability of the solution.

References

- [1] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, January 1989. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/0893608089900208>
- [2] D. C. Psychogios and L. H. Ungar, “A hybrid neural network-first principles approach to process modeling,” *AIChE Journal*, vol. 38, no. 10, pp. 1499–1511, October 1992. [Online]. Available: <https://aiche.onlinelibrary.wiley.com/doi/10.1002/aic.690381003>
- [3] M. W. M. G. Dissanayake and N. Phan-Thien, “Neural-network-based approximations for solving partial differential equations,” *Communications in Numerical Methods in Engineering*, vol. 10, no. 3, pp. 195–201, March 1994. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1002/cnm.1640100303>
- [4] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, September 1998. [Online]. Available: <http://ieeexplore.ieee.org/document/712178/>
- [5] M. Kumar and N. Yadav, “Multilayer perceptrons and radial basis function neural network methods for the solution of differential equations: A survey,” *Computers & Mathematics with Applications*, vol. 62, no. 10, pp. 3796–3811, November 2011. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0898122111007966>
- [6] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015. [Online]. Available: <https://www.nature.com/articles/nature14539>
- [7] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, “Tensorflow: A system for large-scale machine learning,” in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, 2016, pp. 265–283.
- [8] N. Ketkar, “Introduction to keras,” in *Deep Learning with Python: A Hands-On Introduction*. Springer, 2017, pp. 97–111.
- [9] R. Kondor and S. Trivedi, “On the generalization of equivariance and convolution in neural networks to the action of compact groups,” *arXiv preprint arXiv:1802.03690*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.03690>
- [10] P. R. Vlachas, W. Byeon, Z. Y. Wan, T. P. Sapsis, and P. Koumoutsakos, “Data-driven forecasting of high-dimensional chaotic systems with long short-term memory networks,” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 474, no. 2213, p. 20170844, May 2018. [Online]. Available: <http://royalsocietypublishing.org/rspa/article/56754>
- [11] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, February 2019. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125>
- [12] E. Haghighat and R. Juanes, “Sciann: A keras/tensorflow wrapper for scientific computations and physics-informed deep learning using artificial neural networks,” *Computer Methods in Applied Mechanics and Engineering*, vol. 373, p. 113552, January 2021. [Online]. Available: <http://arxiv.org/abs/2005.08803>

- [13] Q. Zhu, Z. Liu, and J. Yan, “Machine learning for metal additive manufacturing: Predicting temperature and melt pool fluid dynamics using physics-informed neural networks,” *arXiv preprint arXiv:2008.13547*, September 2020. [Online]. Available: <http://arxiv.org/abs/2008.13547>
- [14] H. Xu, W. Zhang, and Y. Wang, “Explore missing flow dynamics by physics-informed deep learning: The parameterized governing systems,” *Physics of Fluids*, vol. 33, no. 9, p. 095116, September 2021. [Online]. Available: <https://pubs.aip.org/pof/article/33/9/095116/1064746/Explore-missing-flow-dynamics-by-physics-informed>
- [15] R. Tanios, “Physics informed neural networks in computational finance: High dimensional forward & inverse option pricing,” Master’s thesis, ETH Zurich, June 2021. [Online]. Available: <http://hdl.handle.net/20.500.11850/491555>
- [16] S. Savović, M. Ivanović, and R. Min, “A comparative study of the explicit finite difference method and physics-informed neural networks for solving the burgers’ equation,” *Axioms*, vol. 12, no. 10, p. 982, October 2023. [Online]. Available: <https://www.mdpi.com/2075-1680/12/10/982>
- [17] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, “Scientific machine learning through physics-informed neural networks: Where we are and what’s next,” *Journal of Scientific Computing*, vol. 92, no. 3, p. 88, September 2022. [Online]. Available: <https://link.springer.com/10.1007/s10915-022-01939-z>
- [18] L. Wang and Z. Yan, “Data-driven rogue waves and parameter discovery in the defocusing nonlinear schrödinger equation with a potential using the pinn deep learning,” *Physics Letters A*, vol. 404, p. 127408, July 2021. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0375960121002723>
- [19] S. Hotta, *Mathematical Physical Chemistry: Practical and Intuitive Methodology*, 2nd ed. Singapore: Springer Singapore, 2020.
- [20] G. Mezzetti, “Introduction to the schrödinger equation and an application to the stock market,” Master’s thesis, Luiss Guido Carli University, Rome, Italy, 2021, supervisor: Fausto Gozzi. [Online]. Available: <https://tesi.luiss.it/33247/>
- [21] R. Becerril, F. S. Guzmán, A. Rendón-Romero, and S. Valdez-Alvarado, “Solving the time-dependent schrödinger equation using finite difference methods,” *Revista Mexicana de Física E*, vol. 54, no. 2, pp. 120–132, 2008.
- [22] I. W. Sudiarta and D. J. W. Geldart, “Solving the schrödinger equation using the finite difference time domain method,” *Journal of Physics A: Mathematical and Theoretical*, vol. 40, no. 8, pp. 1885–1900, 2007.
- [23] H. Han, “A finite-difference method for the one-dimensional time-dependent schrödinger equation on unbounded domain,” *Journal of Computational and Applied Mathematics*, 2007.
- [24] A. Kabir, “Numerical simulation of the time-dependent schrödinger equation using the crank-nicolson method,” May 2024.
- [25] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2007.

- [26] J. N. Reddy, *Computational Methods in Engineering: Finite Difference, Finite Volume, Finite Element, and Dual Mesh Control Domain Methods*. Boca Raton, FL, USA: CRC Press, 2024.
- [27] J. Hlongwane, “Algebraic solution of the quantum harmonic oscillator,” *International Journal of Research and Innovation in Applied Science*, vol. 9, no. 4, pp. 62–75, April 2024. [Online]. Available: <https://doi.org/10.51584/IJRIAS.2024.90405>
- [28] M. Rushka and J. K. Freericks, “A completely algebraic solution of the simple harmonic oscillator,” *American Journal of Physics*, vol. 88, no. 11, pp. 976–985, November 2020. [Online]. Available: <https://doi.org/10.1119/10.0001702>